

目录

1.传统目标检测方法	1
1.1 Viola-Jones (人脸检测)	1
1.2 HOG+SVM (行人检测、Opencv 实现)	2
1.3 DPM (物体检测)	2
2.One stage	3
2.1 Yolo 体系变化	3
2.2 YOLO 网络结构是怎样的?	4
2.3 YOLO 的输入、输出、损失函数分别是什么?	4
2.4 YOLO 怎样预测?	4
2.5 YOLOv3 不使用 Softmax 对每个框进行分类的原因?	4
2.6 YOLOv3 的架构	4
2.7 YOLOv4 的架构	5
BackBone 训练策略	6
检测头训练策略	6
检测头推理策略	6
2.8 YOLOv5 的架构	7
2.8.1 yolo5 原理	7
2.8.2 yolo5 引入了 CSP 结构, 介绍一下它的原理和作用?	7
2.9 为什么 SSD 对小目标检测效果不好?	7
2.10 Anchor	7
2.10.1 为什么引入 Anocher?	7
2.10.2 Anocher 为什么要使用不同尺寸和长宽比?	7
2.10.3 Anocher Box 的尺寸该怎么选择?	7
3.Two stage	7
Faster R-CNN	8
Faster RCNN 的 loss 有哪些? 分别讲下.	8
Mask RCNN	8
ROI	9
ROI Pooling	9
目标检测回归损失函数有哪些?	9
One Stage Vs Two Stage	10
4.RPN	10
5.MTCNN	11
6.算法的比较	11
6.1 优缺点	11
6.2 性能评价指标	12
7.人脸检测相关问题	13
7.1 目前主要有人脸检测方法分类?	13
7.2 如何检测图片中不同大小的人脸?	13
7.3 如何设定算法检测最小人脸尺寸?	13
7.4 如何定位人脸的位置?	14
7.5 如何通过一个人脸的多个框确定最终人脸框位置?	14
7.6 如何理解端到端?	14
7.7 目标检测技巧有哪些?	14
8.分类、目标检测、语义分割、实例分割的区别	14
9.做目标检测的话, 数据集用什么?	14
9.1 用自己的数据集做目标检测, 在采集数据时应注意哪些问题?	14

1. 传统目标检测方法

- Viola-Jones
- HOG+SVM
- DPM



1.1 Viola-Jones (人脸检测)

- Haar特征抽取
 - value=白-黑
- 训练人脸分类器 (Adaboost 算法等)
 - ❑ 初始化样本的权重 ω ，样本权重之和为1
 - ❑ 训练弱分类器
 - ❑ 更新样本权重
 - ❑ 循环第2步
 - ❑ 结合做个分类器结果，进行投票

<https://blog.csdn.net/cc13186851239>

➤ 滑动窗口

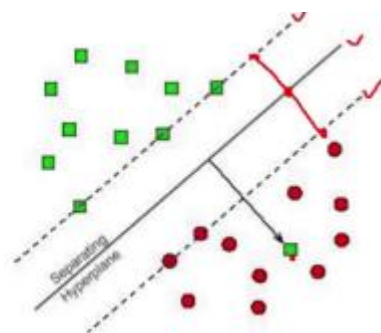
1.2 HOG+SVM (行人检测、Opencv 实现)

➤ HOG特征

- ❑ 灰度化+Gamma变换
- ❑ 计算梯度map $\propto \gamma \theta < \frac{\pi}{360}$
- ❑ 图像划分成小的cell，统计每个cell梯度直方图 18
- ❑ 多个cell组成一个block，特征归一化 4×18
- ❑ 多个block串联，并归一化

➤ SVM

➤ 最大分类间隔面



- 提取HOG特征
- 训练SVM分类器
- 利用滑动窗口提取目标区域，进行分类判断
- NMS
- 输出检测结果

1.3 DPM (物体检测)

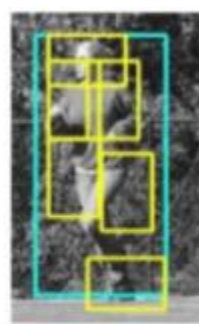
- 计算DPM特征图
- 计算响应图 (root filter和part filter)
- Latent SVM分类器训练
- 检测识别



HOG & Linear SVM



Deformable Parts & Latent SVM



NMS (非极大值抑制算法)

- 目的：为了消除多余的框，找到最佳的物体检测的位置
- 思想：选取那些邻域里分数最高的窗口，同时抑制那些分数低的窗口
- Soft-NMS



- 相邻区域内的检测框的分数进行调整而非彻底抑制，从而提高了高检索率情况下的准确率
- 在低检索率时仍能对物体检测性能有明显提升

1 利用得分，给检测框排序：

选出最大得分的检测框 A1，

将与 A1 的 IOU 重叠率高的检测框，进行删除，

2 其余的剩下的检测再次排序，选择最大的 A2，

将与 A2 的 IOU 重叠率高的检测框，再次进行删除，

3 ...迭代多次

2.One stage

One-stage 核心组件： CNN 网络 + 回归网络

- 使用CNN卷积特征
- 直接回归物体的类别概率和位置坐标值（无region proposal）
- 准确率低、速度相对two-stage快

CNN网络设计原则：

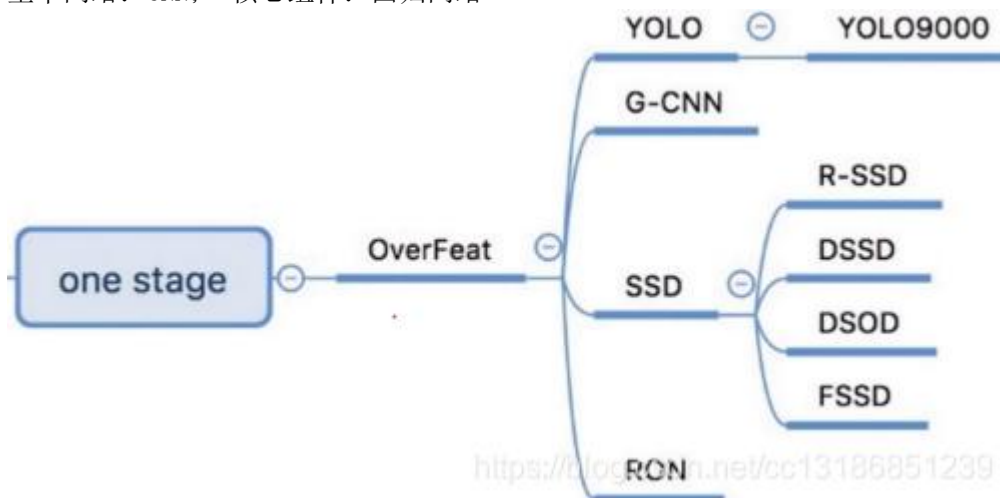
从简到繁再到简的卷积神经网络；
多尺度特征融合网络；
更轻量级的CNN网络。

回归网络：

回归区域（置信度、位置、类别）；
Anchor机制（SSD）。

主要包含 SSD, YOLOv1、YOLOv2、YOLOv3 系列。

主干网络：CNN； 核心组件：回归网络



2.1 Yolo 体系变化

	创新点	相关介绍
SSD	1. 基于Faster R-CNN中的Anchor，提出了相似的先验框（Prior box）； 2. 从不同比例的特征图（多尺度特征）中产生不同比例的预测，并明确地按长宽比分离预测。	
DSSD	1. Backbone: 将ResNet替换SSD中的VGG网络，增强了特征提取能力； 2. 添加了Deconvolution层，增加了大量上下文信息。	
Yolo V1	1. 将整张图作为网络的输入，直接在输出层回归bounding box的位置和所属的类别；2. 速度快	Yolo V1将物体检测任务直接当作回归问题来处理，将候选区和检测两个阶段合二为一。只需一眼就能知道每张图像中有哪些物体以及物体的位置事实上，YOLO也并没有真正的去掉候选区，而是直接将输入图片划分成7x7=49个网格，每个网格预测两个边界框，一共预测49x2=98个边界框。
Yolo V2	YoloV2相对v1版本，在继续保持处理速度的基础上，从预测更准确，速度更快，识别对象更多这三个方面进行了改进。	YOLO9000是在YOLOv2的基础上提出的一种联合训练方法，可以检测超过9000个类别的模型
Yolo V3	1. 多尺度预测（引入FPN）； 2. 更好的基础分类网络（darknet-53，类似于ResNet引入残差结构）； 3. 分类器不再使用Softmax，分类损失采用（二分类交叉损失熵）	模型复杂了不少，可以通过改变模型结构的大小来权衡速度与精度。 YOLOv3 的先验检测系统将分类器或定位器重新用于
Yolo V4	1. 输入端：这里指的是创新主要是训练时对输入端的改进，主要包括Mosaic数据增强、cmBN、SAT自对抗训练； 2. Backbone主干网络：将各种新的方式结合起来，包括：CSPDarknet53、Mish激活函数、Dropblock； 3. Neck：目标检测网络在BackBone和最后的输出层之间往往会插入一些层，比如Yolov4中的SPP模块、FPN+PAN结构； 4. Prediction：输出层的锚框机制和Yolov3相同，主要改进的是训练时的损失函数CIoU_Loss，以及预测框筛选的nms变为DIOU_nms。	YOLOv4框架 = CSPDarknet53 + SPP + PAN + YOLOv3 Backbone: CSPDarknet53 Neck: SPP, PAN Head: YOLOv3
Yolo V5	YOLOv5 的表现要优于谷歌开源的目标检测框架 EfficientDet；YOLOv5 宣称自己速度非常快，有非常轻量级的模型大小，同时在准确度方面又与 YOLOv4 基准相当；大家对YOLOv5命名是争议很大，因为YOLOv5相对于YOLOv4来说创新性的地方很少。	

解释：cmBN: Conv + Mish + BatchNorm

2.2 YOLO 网络结构是怎样的？

强烈推荐阅读：Yolo 发展史

YOLO 网络借鉴了 GoogLeNet 分类网络结构，不同的是 YOLO 使用 1x1 卷积层和 3x3 卷积层替代 inception module。如下图所示，整个检测网络包括 24 个卷积层和 2 个全连接层。其中，卷积层用来提取图像特征，全连接层用来预测图像位置和类别概率值。

2.3 YOLO 的输入、输出、损失函数分别是什么？

YOLO 将输入图像分成 7x7 的网格，最后输出是 7x7xk 的张量。YOLO 网络最后接了两个全连接层，全连接层要求输入是固定大小的，所以 YOLO 要求输入图像有固定大小，论文中作者设计的输入尺寸是 448x448。

2.4 YOLO 怎样预测？

采用 NMS 算法从输出结果中提取最有可能的对象和其对应的边界框。

手写 NMS:

<https://zhuanlan.zhihu.com/p/75348108>

NMS 步骤如下：

1. 设置一个 Score 的阈值，一个 IOU 的阈值；
2. 对于每类对象，遍历属于该类的所有候选框，
 - ①过滤掉 Score 低于 Score 阈值的候选框；
 - ②找到剩下的候选框中最大 Score 对应的候选框，添加到输出列表；
 - ③进一步计算剩下的候选框与②中输出列表中每个候选框的 IOU，若该 IOU 大于设置的 IOU 阈值，将该候选框过滤掉，否则加入输出列表中；
 - ④最后输出列表中的候选框即为图片中该类对象预测的所有边界框
3. 返回步骤 2 继续处理下一类对象。

2.5 YOLOv3 不使用 Softmax 对每个框进行分类的原因？

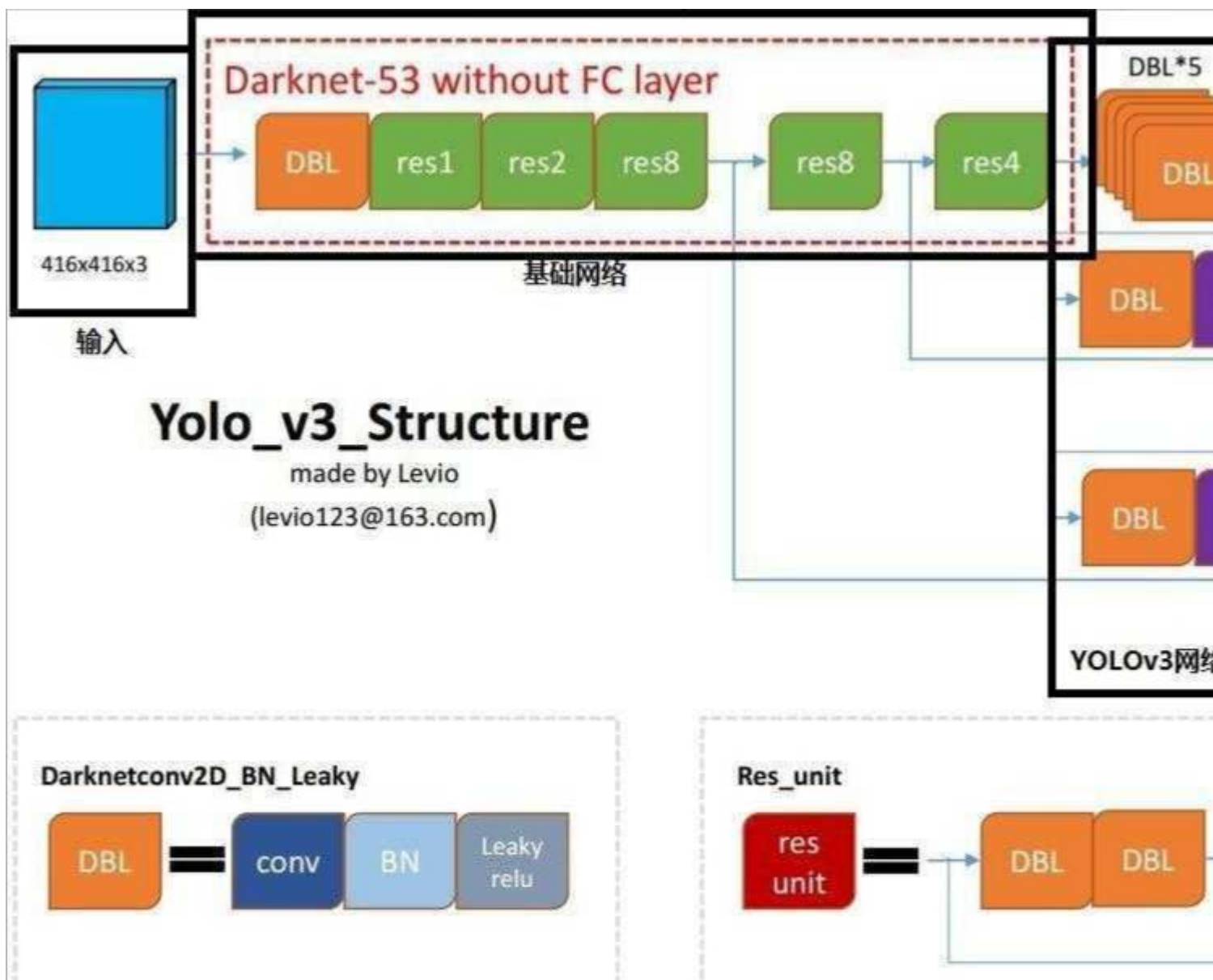
Softmax 使得每个框分配一个类别（score 最大的一个），而对于 Open Images 这种数据集，目标可能有重叠的类别标签，因此 Softmax 不适用于多标签分类。

Softmax 可被独立的多个 logistic 分类器替代，且准确率不会下降。

2.6 YOLOv3 的架构

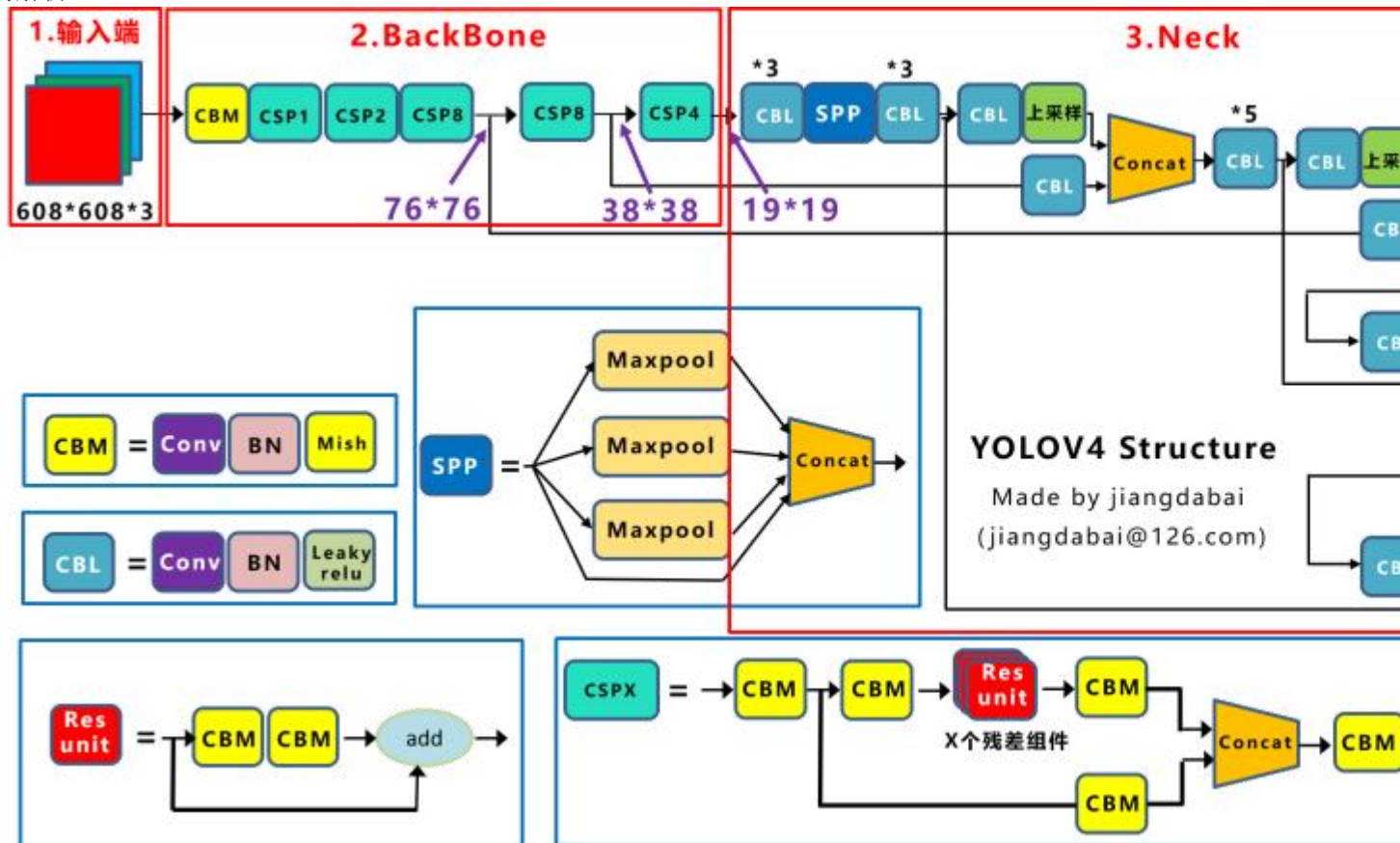
YOLOv3 在之前 Darknet-19 的基础上引入了残差块，并进一步加深了网络，改进后的网络有 53 个卷积层，取名为 Darknet-53。

强烈推荐阅读：yolo3 深度解析



图片引用：YOLOv3 网络结构图

2.7 YOLOv4 的架构



图片若侵权：可联系删除

CSPDarknet53

解决了其他大型 CNN 框架 Backbone 中网络优化的梯度信息重复问题，将梯度的变化从头到尾地集成到特征图中，因此减少了模型的参数量和 FLOPS(每秒浮点运算次数)数值，既保证了推理速度和准确率，又减小了模型尺寸。

BackBone 训练策略

1. 采用了 Mosaic 数据增强

(参考了 CutMix 数据增强；区别:Mosaic 是一种将 4 张训练图像合并成一张进行训练的数据增强方法(而不是 CutMix 中的 2 张)。

每个小批包含一个大的变化图像(4 倍)，减少了估计均值和方差的时需要大 mini-batch 的要求，降低了训练成本。

2. DropBlock 正则化(引入：克服 Dropout 随机丢弃特征)：块的相邻相关区域中丢弃特征。

3. 类标签平滑

检测头训练策略

1. CIoU-loss: CIoU 只是在 DIoU 基础上增加了一项 av

2. CmBN 策略：只在每个 Batch 内部使用 CBN 的方法

3. SAT:

第一阶段：NN 改变原始图像而不是网络权值。通过这种方式，NN 对其自身进行一种对抗式的攻击，改变原始图像，制造图像上没有目标的假象；

第二阶段：训练 NN 对修改后的图像进行正常的目标检测。

消除网格敏感度

4. 单目标使用多 Anchor

5. 余弦模拟退火

检测头推理策略

Mish, SPP, PAN, SAM 和 DIoU-NMS

为了增大感受野，使用了 SPP-block(对任意尺寸的特征图直接进行固定尺寸的池化-> 固定数量的特征)，使用 PAN 代替 FPN 进行参数聚合以适用于不同 level 的目标检测。

SPP：用来解决不同尺寸的特征图如何进入全连接层的

PAN：代替 FPN 进行参数聚合以适用于不同 level 的目标检测, YOLOv4 算法将融合的方法由 addition 改为

Concatenation

DarkNet-53 相对 DarkNet-19 的有如下几点改进：

1. 加深网络层，精度提升，但是速度有所下降，

2. 进入了残差网络 resNet 模块，防止梯度下降；
3. 用卷积 stride=2 代替了池化，防止信息丢失
4. 使用 SPP 实现多尺寸输入，同尺寸输出。

名词解释：

CSPNet（跨阶段局部网络）

SPP-Net（空间金字塔池化网络）：Spatial Pyramid Pooling Networks

PAN（路径聚合网络）：Path Aggregation Network

SAT（自对抗训练）：Self-Adversarial Training

2.8 YOLOv5 的架构

2.8.1 yolo5 原理

原理可以分为四部分：输入端、backbone、Neck、输出端；

输入端：针对小目标的检测，沿用了 v4 的 **mosaic** 增强，当然这个也是 v5 作者在他复现的 v3 上的原创，对不同的图片进行随机缩放、裁剪、排布后进行拼接；二是自适应锚框计算，在 v3、v4 中，初始化锚框是通过对 coco 数据集的进行聚类得到，v5 中将锚框的计算加入了训练的代码中，每次训练时，自适应的计算不同训练集中的最佳锚框值；

backbone：沿用了 V4 的 **CSPDarkNet53** 结构，但是在图片输入前加入了 **Focus** 切片操作，CSP 结构实际上就是基于 **Densnet** 的思想，复制基础层的特征映射图，通过 dense block 发送到下一个阶段，从而将基础层的特征映射图分离出来。这样可以有效缓解梯度消失问题，支持特征传播，鼓励网络重用特征，从而减少网络参数数量。在 V5 中，提供了**四种不同大小的网络结构**：s、m、l、x，通过 **depth**（深度）和 **width**（宽度）两个参数控制。

Neck：采用了 **SPP+PAN** 多尺度特征融合，PAN 是一种自下而上的特征金字塔结构，是在 FPN 的基础上进行的改进，相对于 FPN 有着更好的特征融合效果。

输出端：沿用了 V3 的 head，使用 **GIoU** 损失进行边框回归，输出还是三个部分：**置信度、边框信息、分类信息**。

2.8.2 yolov5 引入了 CSP 结构，介绍一下它的原理和作用？

CSP 结构是一种思想，它和 ResNet、DenseNet 类似，可以看作是 DenseNet 的升级版，它将 feature map 拆成两个部分：

一部分进行**卷积操作**；另一部分和上一部分卷积操作的结果进行 **concat**。主要解决了三个问题：

1. 增强 CNN 的学习能力，能够在轻量化的同时保持着准确性；
2. 降低计算成本；
3. 降低内存开销。

CSPNet 改进了**密集块**和**过渡层**的信息流，优化了**梯度反向传播**的路径，**提升了网络的学习能力**，同时在处理**速度**和**内存**方面提升了不少。

2.9 为什么 SSD 对小目标检测效果不好？

小目标对应的 anchor 比较少，其对应的 feature map 上的 pixel 难以得到训练，这也是为什么 SSD 在 augmentation 之后精确度上涨（因为 crop 之后小目标就变为大目标）要检测小目标需要足够大的 feature map 来提供精确特征，同时也需要足够的语义信息来与背景作区分。

2.10 Anchor

定义：提前在图像上预设好的不同大小，不同长宽比的框；同一位置设置多个不同尺度先验框

2.10.1 为什么引入 Anchor？

使得模型更容易学习。使用 anchor boxes 之后，YOLOv2 的召回率大大提升，所以在 Yolo 之后的版本中，均保留了先验框。

2.10.2 Anchor 为什么要使用不同尺寸和长宽比？

为了得到更大的交并比

2.10.3 Anchor Box 的尺寸该怎么选择？

在 SSD、Faster-RCNN 中，设计了 9 个不同大小和宽高比的 anchor；

anchor box 的选择主要有三种方式：

1. 人为经验选取；
2. k-means 聚类；
3. 作为超参数进行学习

YOLOv2 中建议使用 K-means 聚类来代替人工设计，通过对训练集的 bbox 进行聚类，自动生成一组更加适合数据集的 anchor，可以使网络的检测效果更好。

虽然设置更多的先验框，IOU 能获得一定的提升，但模型的复杂度随之增加，YOLOv2 的作者选择了 K=5 个先验框。

3. Two stage

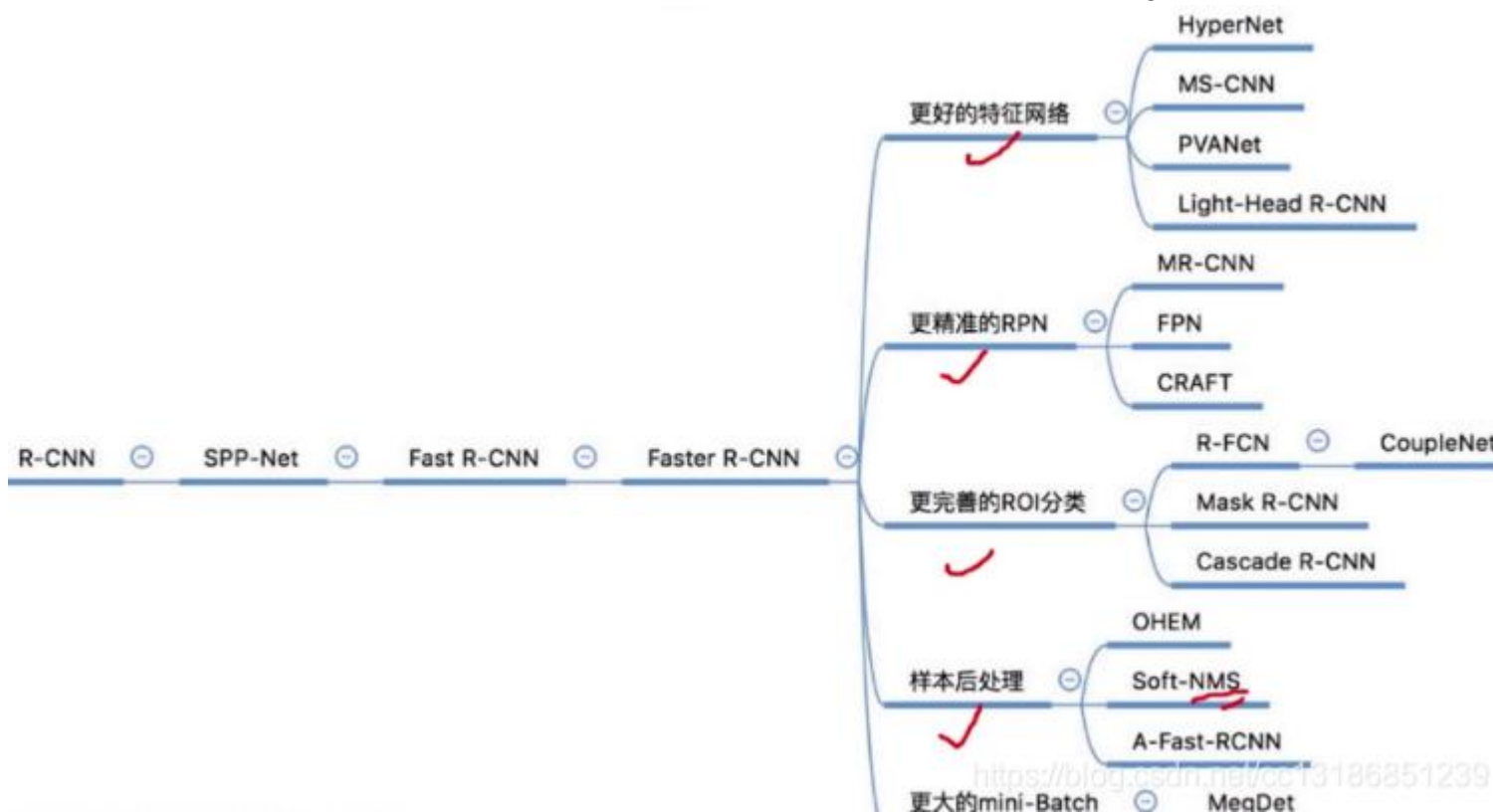
概念：先由算法生成一系列作为样本的候选框，再通过卷积神经网络进行样本分类。

主要是 R-CNN，SPP 系列，Fast R-CNN，Faster R-CNN 等；

目标检测方法概述：

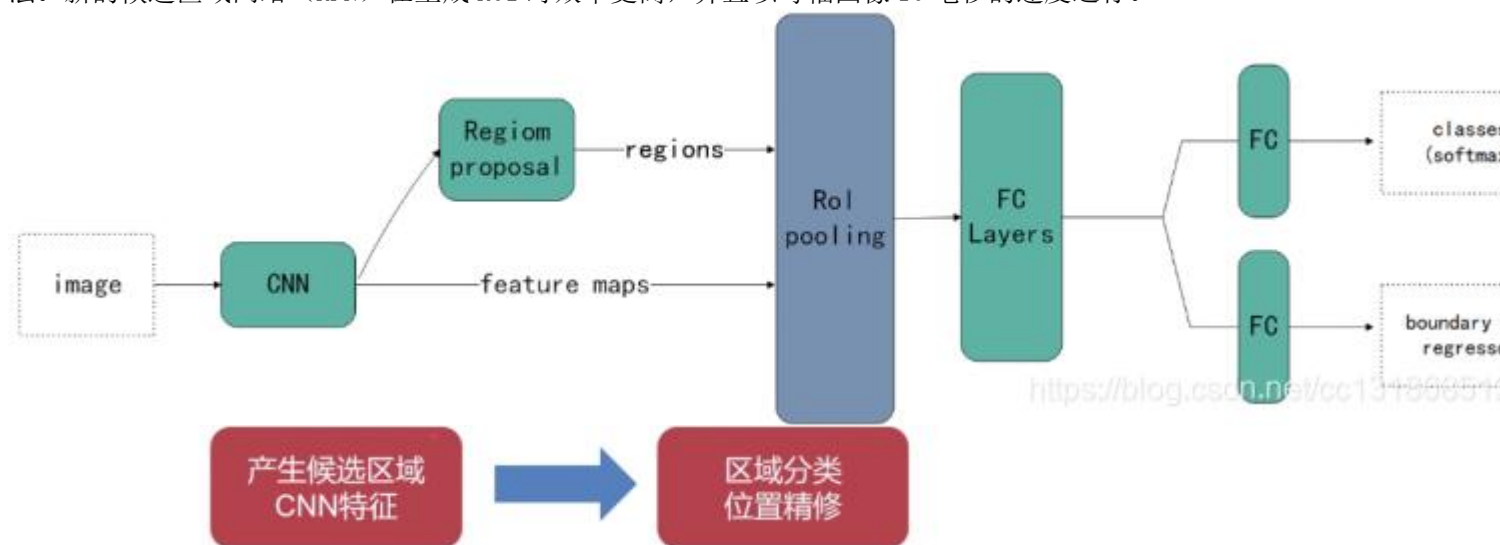
主要通过一个 CNN 来完成目标检测过程，其提取的是 CNN 卷积特征，在训练网络时，其主要训练两个部分，第一

步是训练 RPN 网络，第二步是训练目标区域检测的网络。网络的准确度高、速度相对 One-stage 慢。



Faster R-CNN

Fast R-CNN 依赖于外部候选区域方法，如选择性搜索。在测试中，Fast R-CNN 需要 2.3 秒来进行预测，其中 2 秒用于生成 2000 个 ROI。Faster R-CNN 采用与 Fast R-CNN 相同的设计，只是它用内部深层网络代替了候选区域方法。新的候选区域网络（RPN）在生成 ROI 时效率更高，并且以每幅图像 10 毫秒的速度运行。



Faster RCNN 的 loss 有哪些？分别讲下。

第一部分为分类损失，第二部分为回归损失

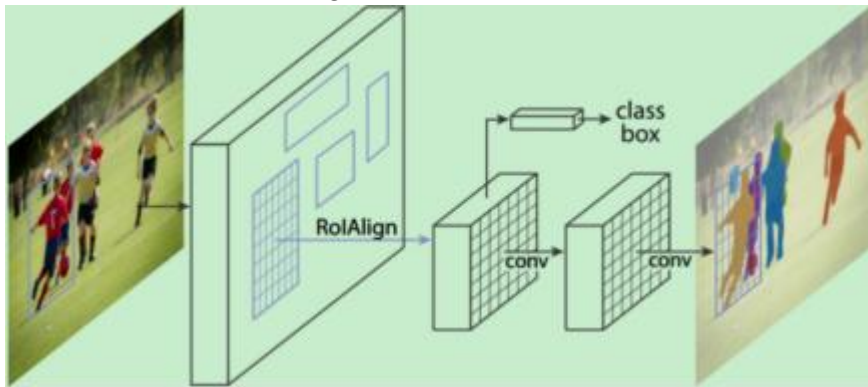
Mask RCNN

创新点：1. Backbone: ResNeXt-101 + FPN

2. RoI Align 替换 RoI Pooling

是一个实例分割算法，主要是在目标检测的基础上再进行分割。算法主要是 Faster R-CNN+FCN，更具体一点就是

ResNeXt + RPN + RoI Align + Fast R-CNN + FCN。

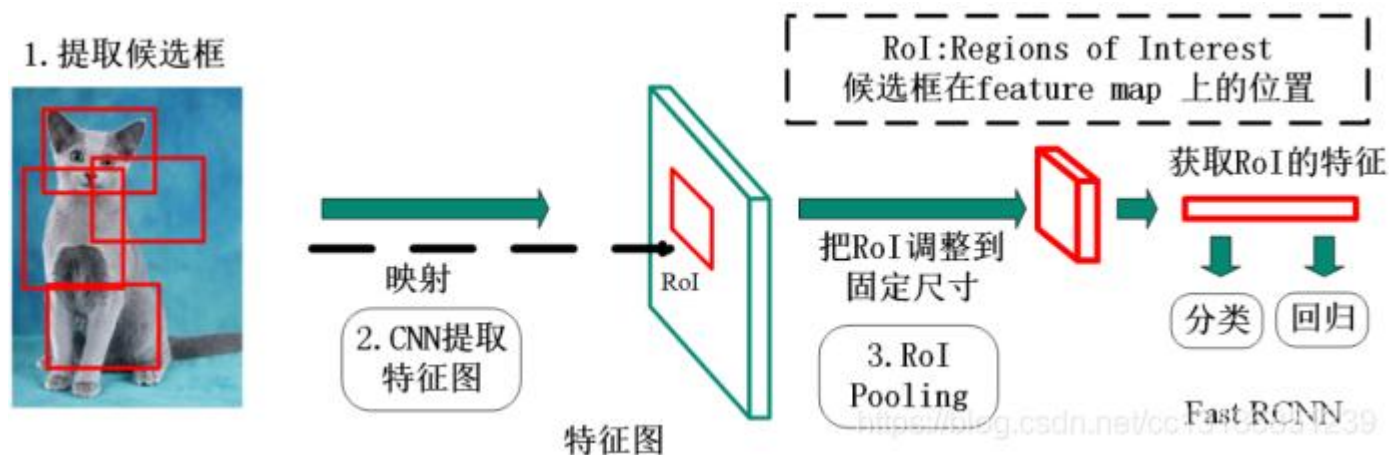


Mask R-CNN 算法步骤:

- 输入一幅你想处理的图片，然后进行对应的预处理操作，或者预处理后的图片；
- 将其输入到一个预训练好的神经网络中（ResNeXt 等）获得对应的 feature map；
- 对这个 feature map 中的每一点设定预定个的 RoI，从而获得多个候选 RoI；
- 将这些候选的 RoI 送入 RPN 网络进行二值分类（前景或背景）和 BB 回归，过滤掉一部分候选的 RoI；
- 对这些剩下的 RoI 进行 RoI Align 操作；
- 对这些 RoI 进行分类（N 类别分类）、BB 回归和 MASK 生成（在每一个 RoI 里面进行 FCN 操作）。

ROI

RoI 是 Region of Interest，一般是指图像上的区域框，但这里指的是由 Selective Search 提取的候选框。往往经过 RPN 后输出的不止一个矩形框，所以这里我们是对多个 RoI 进行 Pooling。



ROI Pooling

目的是对非均匀尺寸的输入执行最大池化以获得固定尺寸的特征图。

我们获得的 anchor 尺寸是不同的，能直接对其全连接，应该将区域提案划分为相等大小的部分（其数量与输出的维度相同）

；然后进行 max pooling.

ROI Polling Vs ROI Align

ROI Pool 存在两次量化误差，首先是将候选框边界量化为整数点坐标值，其次是将量化后的边界区域平均分割成 $k \times k$ 个单元，对每一个单元的边界进行量化。ROI Align 通过双线性插值避免了量化操作，保存了原始 ROI 的空间分布，有效避免了误差的产生；对于检测图片中大目标物体时，两种方案的差别不大，而如果是图片中有较多小目标物体需要检测，则优先选择 ROI Align，更精准一些。

目标检测回归损失函数有哪些？

演进路线是：

Smooth L1 Loss --> IoU Loss --> GIoU Loss --> DIOU Loss --> CIOU Loss

GIoU loss 仍然存在收敛速度慢、回归不准等问题

目标检测中 IOU 是如何计算的？ 检测结果与 Ground Truth 的交集比上它们的并集。

loss	描述	公式	图片
IoU		$IoU = \frac{ A \cap B }{ A \cup B }$	
GIoU	A框与B框计算GIoU时，先找到能将A,B包围的最小框C。 GIoU ≤ IoU, GIoU取值范围-1 到 1. 当A与B没有交集时，IoU=0, 后半部分趋近于1. 当A与B重合时，IoU=1, 后半部分趋近于0.	$GIoU = IoU - \frac{ C \setminus (A \cup B) }{ C }$ 注: $C \setminus (A \cup B)$ 表示 $C - A \cup B$	
DIoU	- 通常基于IoU-based的loss可以定义为 $L = 1 - IoU + R(B, B^{gt})$, 其中 $R(B, B^{gt})$ 定义为预测框 B 和目标框 B^{gt} 的惩罚项。 - DIoU中的惩罚项表示为 $R_{DIoU} = \frac{\rho^2(b, b^{gt})}{c^2}$, 其中 b 和 b^{gt} 分别表示 B 和 B^{gt} 的中心点, $\rho(\cdot)$ 表示欧式距离, c 表示 B 和 B^{gt} 的最小外界矩形的对角线距离, 如下图所示。可以将DIoU替换IoU用于NMS算法当中, 也即论文提出的DIoU-NMS, 实验结果表明有一定的提 • 尺度不变性 • 当两个框完全重合时, $L_{IoU} = L_{GIoU} = L_{DIoU} = 0$, 当2个框不相交时 $L_{GIoU} = L_{DIoU} \rightarrow 2$ • DIoU Loss可以直接优化2个框直接的距离, 比GIoU Loss收敛速度更快 • 对于目标框包裹预测框的这种情况, DIoU Loss可以收敛的很快, 而GIoU Loss此时退化为IoU Loss收敛速度较慢  $L_{DIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2}$		
CIoU	CIoU的惩罚项是在DIoU的惩罚项基础上加了一个影响因子 αv, 这个因子把预测框长宽比拟合目标框的长宽比考虑进去, $R_{CIoU} = \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$, 其中 α 是用于做trade-off的参数, $\alpha = \frac{v}{(1 - IoU) + v}$, v 是用来衡量长宽比一致性的参数, 定义为 $v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2$	$L_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v$	

IoU 为 0 时, 两个框不重叠, 没有交集。

IoU 为 1 时, 两个框完全重叠。

IoU 取值为 0 ~ 1 之间的值时, 代表了两个框的重叠程度, 数值越高, 重叠程度越高。

One Stage Vs Two Stage

➤ 优点

● 速度快

● 避免背景错误, 产生false positives

● 学到物体的泛化特征

➤ 缺点

● 精度低 (定位、检出率)

● 小物体的检测效果不好

➤ 优点

● 精度高 (定位、检出率)

● Anchor机制

● 共享计算量

➤ 缺点

● 速度慢

● 训练时间长

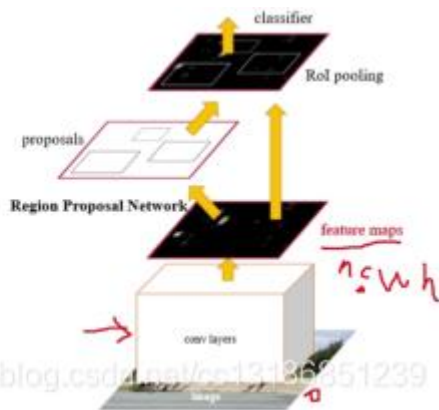
● 误报高

RPN网络

➤ 区域推荐 (Anchor机制)

➤ ROI Pooling

➤ 分类和回归



➤ ROI Pooling

➤ 输入: 特征图、 $rois (1 \times 5 \times 1 \times 1)$ 以及ROI参数

➤ 输出: 固定尺寸的feature map

Roi: 抠图+resize(固定同样尺寸大小)

RPN 网络结构?

生成 anchors $\rightarrow$ softmax 分类器提取 positive anchors $\rightarrow$ bbox reg 回归 positive anchors $\rightarrow$ Proposal Layer 生成 proposals

RPN 执行两种不同类型的预测: 二进制分类和边框回归调整。为了训练, 我们把所有的锚 anchor box 分成两类。一类是「前景」, 它与真实目标重叠并且其 IoU 值大于 0.5; 另一类是「背景」, 它不与任何真实目标重叠或与真实目标的 IoU 值 小于 0.1。

5.MTCNN

流程:

图片经过 Pnet, 会得到 feature map, 通过分类、NMS 筛选掉大部分假的候选;

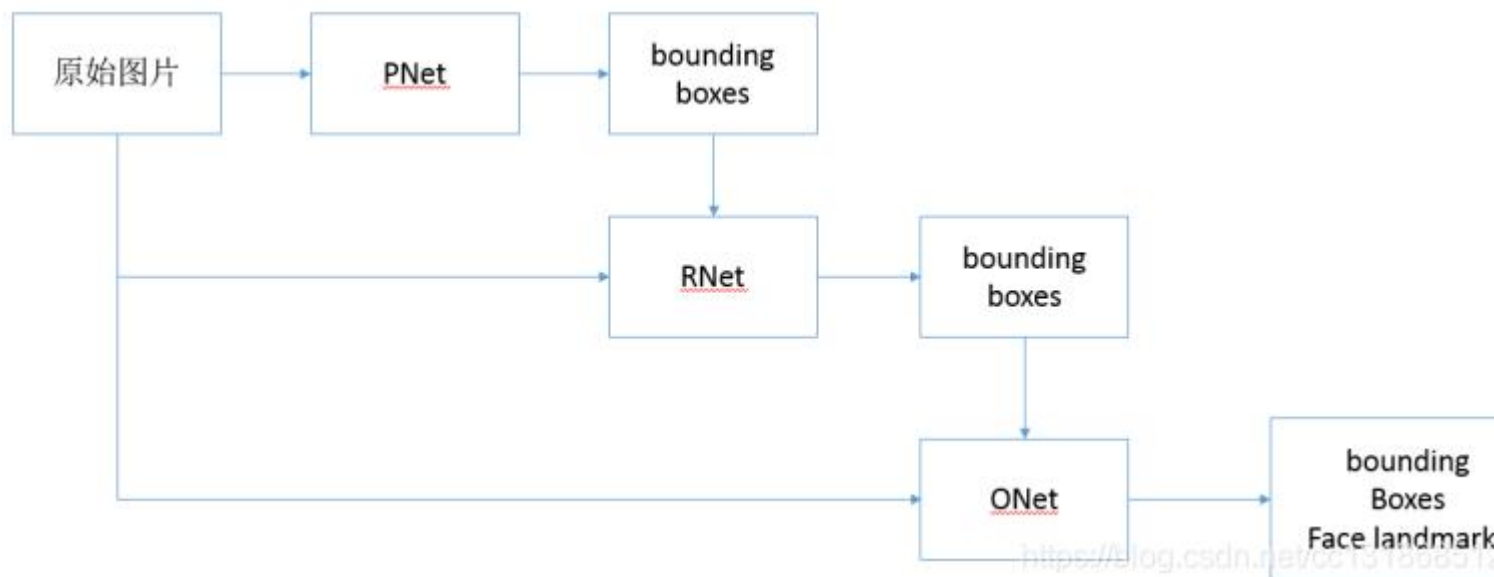
然后剩余候选去原图 crop 图片输入 Rnet, 再对 Rnet 的输出筛选掉 False、NMS 去掉众多的候选;

剩余候选再去原图 crop 出图片再输入到 Onet, 这个时候就能够输出准确的 bbox、landmark 坐标了。

1. 由原始图片和 PNet 生成预测的 bounding boxes。

2. 输入原始图片和 PNet 生成的 bounding box, 通过 RNet, 生成校正后的 bounding box。

3. 输入元素图片和 RNet 生成的 bounding box, 通过 ONet, 生成校正后的 bounding box 和人脸面部轮廓关键点。



6.算法的比较

6.1 优缺点

	优点	缺点
RCNN	最早使用CNN特征	多次使用CNN提取特征、耗时高
SPP-Net	处理任意尺寸的输入图片、共享特征	使用SVM分类
Fast RCNN	引入ROI pooling; bbox回归和cls回归	selective search产生region proposals
Faster RCNN	RPN网络, 学习产生region proposals, 精度高	速度慢、训练时间长
YOLO & YOLOv2	直接回归, 边框定位不够精准; 目标检出率低, 小目标难	误报少、耗时少 https://blog.csdn.net/cc13186851239

6.2 性能评价指标

● 准确率: mAP, 速度: FPS

- TP、FP、TN、FN
- PR曲线
- AP、mAP、
- IoU
- FPS

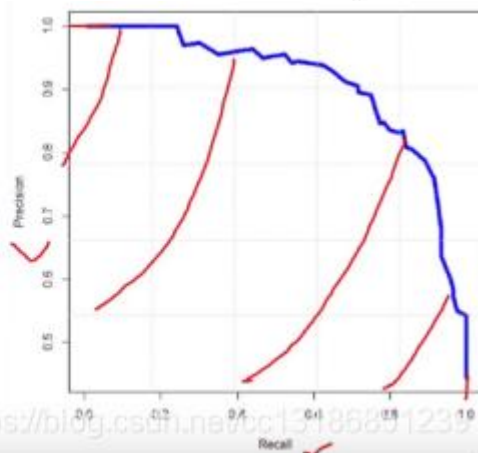
AP 是在单个类别下的, mAP 是 AP 值在所有类别下的均值;

mAP 针对的就是**类识别准确率**, 但是在目标检测任务中还有一个**边框回归任务**, 框的准确率一般用交并比来 IoU 衡量:

$AP/AP50/AP75$ 指的是 $AP^{IoU=0.5:0.05:0.95} / AP^{IoU=0.5} / AP^{IoU=0.75}$,

准确率: mAP, 速度: FPS

- PR曲线
- **Precision**: 查准率, $TP/(TP+FP)$, 该值越大越好, 1为理想状态
- **Recall**: 查全率, $TP/(TP+FN)$, 该值越大越好, 1为理想状态



	Fps	VOC 2007
Overfeat	0.5	
R-CNN ✓	0.077	48-66%
SPP-net ✓		63.1-82.4%
Fast R-CNN ✓		66.9%-70%
Faster R-CNN ✓	15(ZF Model) ✓	73.2%-85.6%
R-FCN ✓	6	83.6%
YOLO ✓	45-150 }	⇒ 58.8%
SSD ✓	58-72 }	75.1%

	Fps	VOC 2007
Overfeat	0.5	
R-CNN ✓	0.077	48-66%
SPP-net ✓		63.1-82.4%
Fast R-CNN ✓		66.9%-70%
Faster R-CNN ✓	15(ZF Model) ✓	73.2%-85.6%
R-FCN ✓	6	83.6%
YOLO ✓	45-150 }	⇒ 58.8%
SSD ✓	58-72 }	75.1%

7.人脸检测相关问题

7.1 目前主要有人脸检测方法分类？

主要包含两个区域：传统人脸检测算法和基于深度学习的人脸检测算法。

传统人脸检测算法主要可以分为4类：基于知识、模型、特征和外观的人脸检测方法；

基于级联 CNN 的人脸检测（cascade cnn）、基于多任务 CNN 的人脸检测（MTCNN）、Facebox 等，很大程度上提高了人脸检测的鲁棒性。

当然通用目标检测算法像 Faster-rcnn、yolo、ssd 等也有用在人脸检测领域，也可以实现比较不错的结果，但是和专门人脸检测算法比还是有差别。

7.2 如何检测图片中不同大小的人脸？

传统人脸检测算法的策略：

- （1）缩放图片的大小
- （2）缩放滑动窗的大小

基于深度学习的人脸检测算法的策略：

- （1）缩放图片大小。（也可以通过缩放滑动窗的方式，基于深度学习的滑动窗人脸检测方式效率会很慢存在多次重复卷积，所以要采用全卷积神经网络（FCN），用 FCN 将不能用滑动窗的方法。）

- （2）通过 anchor box 的方法（通过特征图预测原图的 anchor box 区域）。

7.3 如何设定算法检测最小人脸尺寸？

主要是看滑动窗的最小窗口和 anchorbox 的最小窗口。

- （1）滑动窗的方法

假设通过 12×12 的滑动窗，不对原图做缩放的话，就可以检测原图中 12×12 的最小人脸。但是往往通常给定最小人脸 $a=40$ 、或者 $a=80$ ，以这么大的输入训练 CNN 进行人脸检测不太现实，速度会很慢，并且下一次需求最小人脸 $a=30 \times 30$ 又要去重新训练，通常还会是 12×12 的输入，为满足最小人脸框 a ，只需要在检测的时候对原图进行缩放即可： $w=w \times 12/a$ 。

- （2）anchorbox 的方法

原理类似，这里主要看 anchorbox 的最小 box，可以通过缩放输入图片实现最小人脸的设定。

7.4 如何定位人脸的位置？

1) 滑动窗的方式：基于分类器识别为人脸的框的位置确定最终的人脸；

2) FCN 的方式

通过特征图映射到原图的方式确定最终识别为人脸的位置，特征图映射到原图人脸框是要看特征图相比较于原图有多少次缩放（缩放主要查看卷积的步长和池化层）

3) 通过 anchor box 的方式：

通过特征图映射到图的窗口，通过特征图映射到原图到多个框的方式确定最终识别为人脸的位置。

滑动窗的方式

7.5 如何通过一个人脸的多个框确定最终人脸框位置？

通过 NMS 得到最终的人脸位置：

NMS 改进版本有很多，最原始的 NMS 就是判断两个框的交集，如果交集大于设定的阈值，将删除其中一个框，那么两个框应该怎么选择删除哪一个呢？因为模型输出有概率值，一般会优先选择概率小的框删除。

7.6 如何理解端到端？

就是输入一张图像，直接可以给出检测结果，中间不需要你进行任何操作，很方便。非端到端就是不能一步搞定的，需要分步进行。RCNN 就是典型的非端到端的例子，一张图过来，首先需要利用 SS 等方法提取 Region Proposals，然后送入到神经网络提取特征，最后还要利用 SVM 分类，不能一步到位。

7.7 目标检测技巧有哪些？

1. 数据增强：随机翻转、随机裁剪、添加噪声等也

2. 多尺度训练/测试

通过输入更大、更多尺寸的图片进行训练，能够在一定程度上提高检测模型对物体大小的鲁棒性，仅在测试阶段引入多尺度，也可享受大尺寸和多尺寸带来的增益。

3. 全局语境

把整张图片作为一个 RoI，对其进行 RoI Pooling 并将得到的 feature vector 拼接于每个 RoI 的 feature vector 上，作为一种辅助信息传入之后的 R-CNN 子网络。

4. 预测框微调/投票法

微调法最初是在 SS 算法得到的 Region Proposal 基础上用检测头部进行多次迭代得到一系列 box，在 ResNet 的工作中，作者将输入 R-CNN 子网络的 Region Proposal 和 R-CNN 子网络得到的预测框共同进行 NMS（见下面小节）后处理，

最后，把跟 NMS 筛选所得预测框的 IoU 超过一定阈值的预测框进行按其分数加权的平均，得到最后的预测结果。

投票法可以理解为以顶尖筛选出一流，再用一流的结果进行加权投票决策。

5. 随机权重平均（SWA）

只需快速集合集成的一小部分算力，就可以接近其表现。SWA 可以用在任意架构和数据集上，都会有不错的表现

6. OHEM(在线难例挖掘)

两阶段检测模型中，提出的 RoI Proposal 在输入 R-CNN 子网络前，我们有机会对正负样本（背景类和前景类）的比例进行调整。通常，背景类的 RoI Proposal 个数要远远多于前景类，Fast R-CNN 的处理方式是随机对两种样本进行上采样和下采样，以使每一 batch 的正负样本比例保持在 1:3，这一做法缓解了类别比例不均衡的问题，是两阶段方法相比单阶段方法具有优势的地方，也被后来的大多数工作沿用。

7. Soft NMS(软化非极大抑制)

用于去除重合度（IoU）较高的预测框，只保留预测分数最高的预测框作为检测输出

8. RoI 对齐

采用双线性插值的方法将 RoI 的表示精细化，并带来了较为明显的性能提升

8. 分类、目标检测、语义分割、实例分割的区别

<https://www.cnblogs.com/zxj9487/p/11154316.html>

9. 做目标检测的话，数据集用什么？

COCO 上训练好的模型 fine tune。

PASCALVOC 数据集：有 VOC2007 和 VOC2012 两个数据集，类别仅 20 个，被看成目标检测方向的一个基准数据集。

COCO 数据集：包含 80 个类别

ImageNet 数据集

9.1 用自己的数据集做目标检测，在采集数据时应注意哪些问题？

1. 首先可以找一些公开的数据集比如 VOC 数据集，先跑通实验，能训练和测试成功。确保数据和算法是没有问题的。

2. 其次，制作你的数据集，看是否可以训练成功，然后再去仔细分析测试的结果，去调优。

3. 阅读代码和实现原理，然后去通过调整代码去对算法做一些改进，然后看精度是否有提升。